

Reviewer Pack — Minimal Rank of Quandle Representations Bounds Monotone Circ...

Entry c39905eeffac · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 22:12:06 UTC

Minimal Rank of Quandle Representations Bounds Monotone Circuit Size for k-CLIQUE

Entry ID: c39905eeffac **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 22:12:06 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

[‘For any k-CNF formula F , the minimal rank of the quandle representation Q_F associated with F is at least $\Omega(n^k / k!)$, ‘where n is the number of variables in F .’, ‘Further, for a monotone circuit C computing k-CLIQUE, the size of C is at most $O(2^n / (n^k * k!))$.’]

Rationale (proposer’s reasoning):

[‘Quandle theory provides a combinatorial structure that can encode logical operations, and its representations may capture non-trivial properties of CNF formulas.’, ‘The conjecture proposes that the rank of quandle representations serves as a lower bound for monotone circuit size, potentially providing insights into the complexity of k-CLIQUE.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 990e36b87cf5fae7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all n in $\{1, 2, \dots, 40\}$, the minimal rank of quandle representation Q_F associated with k-CNF formula F is $\geq \Omega(n^k / k!)$ AND the size of monotone circuit C computing k-CLIQUE is $\leq O(2^n / (n^k * k!))$. The conjecture is falsified if either condition fails for any n or seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'quandle theory' AND 'boolean circuit complexity' AND 'monotone circuit size' - 'minimal rank quandle representation' AND 'bounds monotone circuit size' AND 'k-CLIQUE' - ' $\Omega(n^k / k!)$ in quandle theory' AND ' $O(2^n / (n^k * k!))$ for boolean circuits'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        clauses = []
        for _ in range(k):
            clause = set(random.sample(range(1, n+1), 2))
            if random.choice([True, False]):
                clause = {x: -y for x, y in clause.items()}
            clauses.append(clause)
        return clauses

    def quandle_representation(clauses):
        n = max(max(abs(x) for x in clause) for clause in clauses)
        Q = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in clauses:
            for x, y in clause.items():
                Q[x][y] += 1
                Q[y][x] += 1
        return Q

    def matrix_rank(matrix):
        m = len(matrix)
        n = len(matrix[0])
        rank = 0
        for i in range(min(m, n)):
            if matrix[i][i] != 0:
                for j in range(i + 1, m):
                    factor = matrix[j][i] / matrix[i][i]
```

```

        for k in range(n):
            matrix[j][k] -= factor * matrix[i][k]
        rank += 1
    return rank

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def monotone_circuit_size(n, k):
    return math.ceil(2**n / (n**k * factorial(k)))

results = []
for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5): # Ensure at least 30 instances per seed
        k = random.randint(1, min(n, 5))
        F = generate_k_cnf(n, k)
        Q_F = quandle_representation(F)
        rank_Q_F = matrix_rank(Q_F)
        lower_bound = n**k / factorial(k)

        if rank_Q_F < lower_bound:
            return {
                "metric_name": "Minimal Rank of Quandle Representation",
                "metric_value": rank_Q_F,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, k={k}, Q_F_rank={rank_Q_F}, lower_bound={lower_bound}"
            }

        C_size = monotone_circuit_size(n, k)
        upper_bound = 2**n / (n**k * factorial(k))

        if C_size > upper_bound:
            return {
                "metric_name": "Monotone Circuit Size",
                "metric_value": C_size,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, k={k}, C_size={C_size}, upper_bound={upper_bound}"
            }

    return {
        "metric_name": "Minimal Rank of Quandle Representation",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys

```

```

seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
results = []

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result["metric_value"])

mean = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
support_fraction = sum(1 for r in results if r >= 0.9 * mean) / len(results)

if all(r >= 0.9 * mean for r in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
elif any(r < 0.9 * mean for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result < 0.9 * mean)
    print(f"RESULT: FALSIFIED counterexample='n=40' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_635facd4.py", line 107, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_635facd4.py", line 67, in run_trial
    F = generate_k_cnf(n, k)
        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_635facd4.py", line 26, in generate_k_cnf
    clause = {x: -y for x, y in clause.items()}
              ~~~~~^~~~~~
AttributeError: 'set' object has no attribute 'items'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify the conjecture's conditions for all n in $\{1, 2, \dots, 40\}$. | next: Investigate the cause of the crash and rerun the test to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11290
2	propose	ollama_remote	glm4:latest	0	0	10197
3	propose	ollama_remote	glm4:latest	0	0	11657
4	propose	ollama_remote	glm4:latest	0	0	9918
5	preregistration	ollama_remote	glm4:latest	0	0	6736
6	novelty	ollama_remote	glm4:latest	0	0	4972
7	novelty	ollama_remote	glm4:latest	0	0	5463
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19481
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14725
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14011
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12156
12	judge	ollama_remote	glm4:latest	0	0	8709

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129317 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/c39905eeffac.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c39905eeffac.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c39905eeffac.tar.gz` (if generated)